

The Direct Answer

To bypass Android’s low-memory killer and keep your APK lightweight, your weights, indexing databases, and asset codebases must live outside the APK binary as "Device Local" files on your internal storage (/sdcard/ or local file streams), loaded dynamically at runtime. [1]

Your APK should only contain the structural application logic (the UI controllers, foreground service declarations, and NPU JNI bindings). If you bundle models, vector stores, or third-party node repositories directly inside the compiled .apk, you will cause severe memory pressure, resulting in the Android OS immediately terminating your background services. [2]

1. The Native Voice Bug: Why Claude Code Failed Instantly

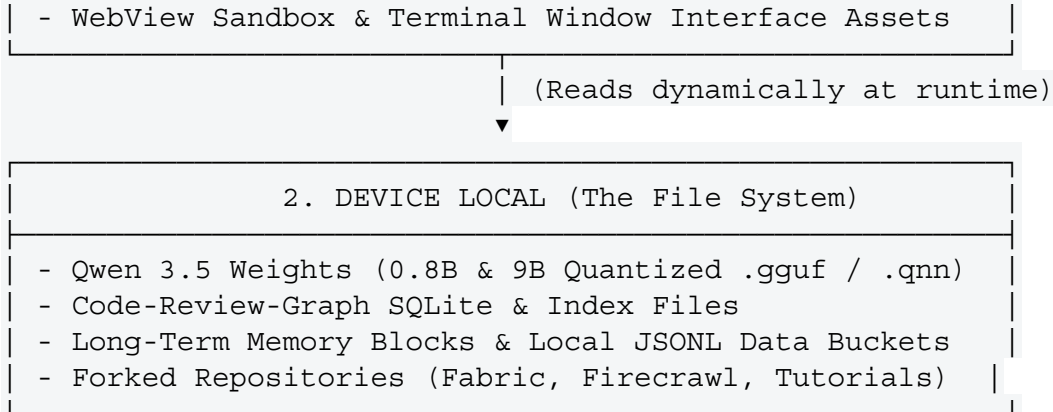
You hit a massive, known wall with Anthropic's **Claude Code /voice dictation interface**. [3, 4]

-
- **The Bug:** The official claude-code core package relies on an automated microphone initialization routine that forces an instant system microphone check at boot. [3]
- **Why it Crashed on Your Device:** Inside an Android APK or a sandboxed terminal environment, a direct node package cannot natively trigger or intercept the Android OS microphone permission loop. The package immediately searches for a missing local compilation or plugin wrapper, encounters an invalid reference, and triggers an unhandled script failure. [5, 6]
- **Your Fix:** Since you are building the voice stack **directly in the APK**, you do not use Claude Code’s voice plugin. Your custom Kotlin layer captures the hardware mic buffer natively via the **Android Media SDK**, handles the Speech-to-Text conversion locally, and feeds Claude the finalized text array inside your meta-prompt wrapper. You bypass their broken internal handler completely.
-

2. APK Local vs. Device Local (The Performance Split)

Keep your application package under 50MB by dividing your assets strictly across these two boundaries:

1. APK LOCAL (The Logic Engine)
- Kotlin UI, Foreground Services, Media/Game SDK Hooks
- Qualcomm QNN / QAI RT SDK Native Library Blobs (.so)



What Stays inside the APK (APK Local)

-
- **The Qualcomm Binary Runtimes:** Your .so (Shared Object) compilation libraries from the **Qualcomm QNN / QAIRT SDK** must be packaged inside the APK's jniLibs/hexagon/ folder. This tells the Android kernel how to talk straight to the NPU chip.
- **The Interface Frames:** Your basic terminal window shell layout, file explorer tree UI, and the isolated WebView sandbox layout framework.
-

What Lives on System Storage (Device Local)

-
- **The Brains (The Models):** Your quantized Qwen 3.5 0.8B and 9B model files live inside a dedicated path on your device storage (e.g., /sdcard/AI/models/). When your APK opens, your code executes a **Load Only** pointer to that file path using memory mapping (mmap). The weights never load into the app's garbage-collected Java heap; they stream straight from storage directly into the hardware NPU.
- **The Code Graphs:** The persistent index databases built by **code-review-graph** and graphify must be written to your app's local device cache folder (context.cacheDir or /sdcard/AI/graphs/).
- **Your Repositories:** All your custom forks (Fabric, Open Interpreter, Firecrawl, MarkTechpost tutorials) live as plain file trees on your local system storage. Your APK uses standard Java file streams to scan and parse their contents on demand.
-

3. Optimizing the Dynamic Loading Pipeline

By configuring your system this way, your APK boots instantly because it has no massive footprint. When a tool call is initiated:

1. Your **Foreground Service** catches your voice command natively.
2. It sends a localized file path parameter pointing to your repository storage block directly to

your NPU pipeline.

3. The model reads only the lines extracted by your local code-review-graph asset, processing the data with maximum **context reduction** while maintaining absolute permission permanence.

This keeps your environment perfectly isolated, incredibly fast, and permanently protected from Android's memory killer loops.

Since you are mapping out your asset paths, do you want to format the exact **Kotlin file-pointer string** that maps your local storage folder directly into your native Qualcomm NPU initialization function?

[1] <https://www.linkedin.com>

[2] <https://www.reddit.com>

[3] <https://code.claude.com>

[4] <https://aivancity.ai>

[5] <https://code.claude.com>

[6] <https://unmarkdown.com>